

Lab 4 Vibe Coding assignment analysis

10/07/2026 19:54:42

<https://claude.ai/chat/43d45a79-72e6-4cf3-a2df-27b2ee3389c4>

https://github.com/SETAPESU26/27_centipede I need to complete my La...

Nothing modified. I read the README and all of `game.py` (the only cod...

Now implement ONLY Task 1 from the README: fix the mushroom...

Task 1 is a one-line change in `game.py`. I tested it on a scratch clone in...

Now implement ONLY Task 2 from the README. Implement the...

Task 2 is done. I verified it on my scratch clone in the sandbox, so your... similarly give task 3 and 4

Tasks 3 and 4 are done. I tested both on my scratch clone in the sandbo...



https://github.com/SETAPESU26/27_centipede

I need to complete my Lab 4 Vibe Coding assignment.

I have provided you with:

1. The Lab 4 assignment handout PDF.
2. The assigned GitHub repository:

https://github.com/SETAPESU26/27_centipede

Please work with the repository I have provided and follow the Lab 4 handout requirements exactly.

IMPORTANT:

- Do NOT immediately modify the code.
- First inspect the repository structure.
- Read and understand the README completely.
- Inspect all relevant Python/game files.
- Identify what the existing game is supposed to do.
- Identify the broken code/bugs that need to be fixed.
- Identify EVERY feature/change requested in the README.
- Do not invent additional features that are not requested.
- Do not make changes to the repository yet.

After inspecting everything, give me a clear report with:

1. What the game currently does.
2. How to run the game.
3. Dependencies required.
4. Existing bugs/problems that need to be fixed.
5. Every feature required by the README, listed individually in the order we should implement them.

6. Which files will likely need to be modified for each task.
7. A recommended sequence of separate tasks/commits so that each task can have its own Git commit.

Remember that this is a Vibe Coding assignment, so I need to implement the fixes/features through prompts and keep the changes separated into individual tasks.

Do NOT modify any files yet. First analyze and explain the repository and the required tasks.

10/7/2026, 12:02:20 PM



Detailing gameplay mechanics and noting a durable fact to save. ✓

🌐 GitHub - SETAPESU26/27_centipede: Implementation of Centipede in Pygame (github.com)

🌐 27_centipede/game.py at main · SETAPESU26/27_centipede (github.com)

🕒 Detailing gameplay mechanics and noting a durable fact to save.

🔑 Updating memory

✓ Done

Nothing modified. I read the README and all of **game.py** (the only code file, 175 lines) statically. I haven't run it, and the page view stripped indentation, so I'm going by the logic.

1. What the game does

A Centipede-lite clone in Pygame (600×630 window, 30×30 grid, 20px cells).

- **Player:** moves with the arrow keys inside the bottom 6 rows, and Space fires one bullet at a time straight up.
- **Centipede:** starts top-right with **10 + wave** segments. It winds left and right, dropping a row at a border or a mushroom. At the bottom it turns around and climbs back up to the player zone.
- **Mushrooms:** up to 45 random ones with 4 HP each. They block the centipede and can be shot.
- **Scoring:** destroying a mushroom gives 5, a head shot gives 100, and a body shot gives 10.
- **Splitting:** a shot body segment turns into a full-HP mushroom and splits the chain in two, each half getting its own head.

- **Lives and waves:** 3 lives, with 1.5s of invulnerability after each respawn. Clearing all segments starts the next wave, which is currently only longer, not faster.

2. How to run

```
pip install pygame python game.py
```

Controls: arrows to move, Space to fire, **R** to reset.

3. Dependencies

- Python 3.10+
- **pygame**
- **random** (standard library)
- No image or sound assets

4. Bug to fix

Where	Problem	Fix
Game.hit_mushroom	It deletes the mushroom when HP ≤ 1 , so HP goes 4→3→2→1 and it is gone on the 3rd hit	Change the condition to ≤ 0

This is the README's "one deliberate bug". I found nothing else broken. The expected behaviors in the README (border and mushroom reversal, head vs body splitting, invulnerability, next wave) all check out in the code.

5. Required tasks, in implementation order

#	Task	What it needs
1	Fix mushroom durability	$\leq 1 \rightarrow \leq 0$
2	Implement mushroom_color(hp)	Return an (r,g,b) per HP, or None for the default
#	Task	What it needs
3	Implement on_segment_hit(segment, score)	Spark effect or bonus on a segment hit; the return value is ignored
4	Implement wave_speed_bonus(wave)	Return a multiplier, e.g. $1 + 0.15 * (\text{wave} - 1)$; None means default speed

Task 1 has to come first. With the bug, a mushroom at HP 1 is never drawn, so you couldn't see Task 2's color for the last HP level.

6. Files to modify

Everything is in `game.py`.

Task	Where
1	<code>hit_mushroom</code> only
2	<code>mushroom_color</code> stub
3	<code>on_segment_hit</code> stub, and possibly small hooks in <code>update</code> and <code>draw</code> (see below)
4	<code>wave_speed_bonus</code> stub only

Two catches in the README ideas

- **Task 2's "flash white when hit"**: the function only receives `hp`, with no hit timing or position. A stateless version can't do a flash. The simplest option is a distinct color per HP level (e.g. green → yellow → orange → red), which fits the stub as written. A real flash would need extra tracking in `hit_mushroom` and `draw`.
- **Task 3's "spark effect"**: the hook gets no access to `self` or the screen, and `draw` wipes the screen every frame. A spark needs a module-level particle list plus tiny additions to `update` and `draw`. A bonus score has the same problem, since the hook can't change `self.score`. The no-edit alternative is a sound or console message, but that's less visible on video.

Decide before prompting whether you're okay touching `update` and `draw` for sparks.

It's still in the spirit of the lab, since these are suggested ideas.

7. Recommended commit sequence

Commit	Content
0	Baseline: original <code>game.py</code> and README in <code>Lab-4/</code> , then record the before video

1	Fix mushroom durability
2	<code>mushroom_color</code>
3	<code>on_segment_hit</code>
4	<code>wave_speed_bonus</code>
5	Add the after video, chat export (PDF/doc) and final cleanup

Tips:

- **Before video:** shoot one mushroom three times and show it vanish.
- **After video:** wave 2 is hard to reach in 10 seconds, so Task 4 may not show on camera. You'll want a plan for that, such as a separate short clip.
- **Chat history:** keep all the implementation prompts in one chat so the export is clean. The handout wants a doc/PDF, while the README says a link, so do both.
- **Remote:** push to your own repo, not SETAPESU26, and don't raise a PR.

Let me know your call on the Task 2 and Task 3 approach, and I'll write the prompts for each task.



Now implement ONLY Task 1 from the README: fix the mushroom durability bug.

Current issue:

`Game.hit_mushroom` removes a mushroom when its HP is `<= 1`, which means a mushroom with 4 HP disappears after only 3 hits.

Change the logic so that the mushroom is removed only when its HP reaches 0.

Requirements:

- Modify only what is necessary for this bugfix.
- Do not implement any of the other README tasks yet.
- Do not add new features.
- Preserve the existing gameplay and scoring.
- Keep the code simple and consistent with the existing style.

After making the change:

1. Show me exactly what you changed and why.
2. Tell me how to test it.
3. Do not make any changes related to mushroom colors, segment-hit effects, or wave speed yet.

10/7/2026, 12:10:55 PM



Figuring out how to implement and deliver the requested code change. ✓

🕒 Figuring out how to implement and deliver the requested code change.

📄 Clone repo, apply one-line fix, show diff, verify hit counts headlessly

✓ Done

Task 1 is a one-line change in `game.py`. I tested it on a scratch clone in my sandbox, so your own clone still needs the edit.

What changed

In `Game.hit_mushroom`:

```
def hit_mushroom(self, cell):    self.mushrooms[cell] -= 1
-   if self.mushrooms[cell] <= 1: +   if self.mushrooms[cell] <= 0:
del self.mushrooms[cell]        self.score += 5
```

Why: HP starts at 4 and drops by 1 per hit. With `<= 1`, the mushroom was deleted when HP hit 1, which is the 3rd hit. With `<= 0`, it's deleted at 0, which is the 4th hit, as the README expects. Scoring is unchanged: +5 only on destruction. I didn't touch anything else.

How to test

Quick check: run `python game.py`, park under a mushroom, and shoot it.

Hit	Expected
1	Still there, slightly darker
2	Still there, darker
3	Still there, darkest (this was the vanishing point before)
4	Gone, score +5

Headless check: I ran this on my scratch copy, and the results matched the table:

Hit	Mushroom exists	HP
1	Yes	3
2	Yes	2
3	Yes	1
4	No	gone

The score went up by 5 in total.

Commit

```
git add game.py
```

```
git commit -m "Task 1: Fix mushroom durability bug (survive 4 hits)"
```

Ready for Task 2 when you are. Pick the approach first: color per HP level, or flash-onhit, which needs extra tracking.

10/7/2026, 12:11:09 PM



Now implement ONLY Task 2 from the README.

Implement the `mushroom_color(hp)` function.

Requirements:

- Return an RGB `(r, g, b)` tuple based on the mushroom's current HP.
- Use a clearly distinguishable color for each HP level.
- The color should visually communicate the mushroom becoming weaker as its HP decreases.
- Return `None` for the default/fallback case if that is how the existing stub is designed.
- Integrate the function into the existing mushroom drawing code so the mushroom actually changes color according to its HP.
- Do not change the mushroom durability logic from Task 1.
- Do not implement segment-hit sparks or wave-speed changes.
- Do not make unrelated changes.

After implementing:

1. Explain exactly which lines/functions you changed.
2. Explain how the color changes with HP.
3. Tell me how to test it in the game.

10/7/2026, 12:15:57 PM



Assigning distinct colors to each mushroom type. ✓

- ⌚ Assigning distinct colors to each mushroom type.
- 📄 Implement mushroom_color, verify drawn pixel colors match per HP
- ✅ Done

Task 2 is done. I verified it on my scratch clone in the sandbox, so your own clone still needs the edit. I rendered mushrooms at each HP level, and the pixel colors drawn on screen matched what the function returns.

What changed

All in `game.py`. Only the stub area changed. The Task 1 line shows in the diff only because my scratch copy had it applied.

```
+MUSHROOM_COLORS = { + 4: (230, 90, 200), # bright pink - full health
+ 3: (220, 60, 120), # rose
+ 2: (190, 40, 50), # red + 1: (110, 20, 30), # dark maroon - about to break
+}
++ def mushroom_color(hp):
+     """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default.""" -
+     pass
+     return MUSHROOM_COLORS.get(hp)
```

- **No drawing code touched:** `draw` already calls `mushroom_color(hp)` or `(default)` for each mushroom, so filling in the function is the whole integration.
- **Fallback:** `.get(hp)` returns `None` for any HP outside 1 to 4, so the original default color still applies there.
- **Task 1 untouched:** `hit_mushroom` is unchanged.

How the color changes

HP	Color	Look
4 (fresh)	(230, 90, 200)	Bright pink
3	(220, 60, 120)	Rose
2	(190, 40, 50)	Red
HP	Color	Look
1	(110, 20, 30)	Dark maroon

It shifts from pink to red to dark maroon, so a mushroom looks more damaged with each hit. I avoided green and yellow on purpose, since the centipede body is green and the head is yellow.

How to test

1. Run `python game.py`.
2. Shoot one mushroom four times.
3. Expect pink → rose → red → maroon, then it disappears on the 4th hit, which also confirms Task 1 still works.
4. Shoot a centipede body segment. The mushroom it leaves behind should appear fresh pink, since it spawns at full HP.

Commit

```
git add game.py
git commit -m "Task 2: Implement mushroom_color with per-HP colors"
```

For Task 3, the sparks need a module-level particle list plus small hooks in `update` and `draw`. Tell me if you'd rather go that way or use a no-edit alternative like a sound.

10/7/2026, 12:16:17 PM



similarly give task 3 and 4

10/7/2026, 12:23:56 PM



Wiring spark updates and drawing into the game loop. ✓

- 🕒 Wiring spark updates and drawing into the game loop.
- 📄 Apply Task 3 on scratch clone and show diff Test Task 3
- 📄 sparks spawn, draw, and expire
- 📄 Apply Task 4 and verify speed scaling by wave
- ✓ Done

Tasks 3 and 4 are done. I tested both on my scratch clone in the sandbox, so your own clone still needs the edits. I went with the spark effect for Task 3, since you didn't pick an approach and it's the README's suggested idea.

Task 3: `on_segment_hit` (spark effect)

The hook gets no screen or game access, and `draw` clears the screen every frame. So the sparks live in a module-level list, with two small helpers and one-line hooks in

`update` and `draw`.

```
+SPARK_LIFE = 0.4 +sparks = [] # each spark is [x, y, vx, vy, time_left]
++ def on_segment_hit(segment, score):
- pass
+ x = segment.col * CELL + CELL // 2 + y = segment.row
* CELL + CELL // 2 + for _ in range(12):
+ velocity = pygame.Vector2(random.uniform(60, 180),
0).rotate(random.uniform(0, 360)) + sparks.append([x, y, velocity.x, velocity.y, SPARK_LIFE])
+
+
```

```

+def update_sparks(dt):
+    for spark in sparks:
+        spark[0] += spark[2] * dt +      spark[1] +=
spark[3] * dt
+        spark[4] -= dt +  sparks[:] = [spark for spark in sparks if spark[4] > 0]
+
++def draw_sparks(screen):
+    for x, y, _, _, life in sparks:
+        fade = life / SPARK_LIFE
+        pygame.draw.circle(screen, (255, int(60 + 180 * fade), int(120 * fade)), (int(x), int(y)), 2)

```

Two one-line hooks inside **Game**:

```

        self.update_bullet(dt)
+    update_sparks(dt)      # in Game.update ...
+    draw_sparks(screen)    # in Game.draw, after the centipede loop      if self.bullet:

```

- **What you see:** a burst of 12 small orange-yellow sparks flies out from the destroyed segment and fades in about 0.4s.
- **Not included:** a head-shot bonus. The hook can't change **self.score**, and it isn't a required feature.

Test results:

Check	Result
Splitting a chain	2 chains, 12 sparks, score +10
Drawing	All 12 sparks showed up as non-background pixels
Expiry	All gone after 9 frames
Check	Result

Real bullet hit

Sparks spawned and the bullet cleared

In-game test: shoot a centipede segment (head or body) and you should see the burst.

```
git add game.py
git commit -m "Task 3: Add spark effect on segment hit"
```

Task 4: `wave_speed_bonus` (one line)

```
def wave_speed_bonus(wave):
- pass
+ return 1 + 0.15 * (wave - 1)
```

Wave	Multiplier	Tick (s)
1	1.00	0.090
2	1.15	0.078
3	1.30	0.069
5	1.60	0.056

Wave 1 returns exactly 1.0, so it plays as before. In a simulated 1-second run, the wave 1 head moved 11 columns and the wave 3 head moved 14.

In-game test: clear a wave and the next centipede should visibly move faster. The HUD shows the wave number.

```
git add game.py
git commit -m "Task 4: Increase centipede speed each wave"
```

Heads-up for the after video

Wave 2 is hard to reach in 10 seconds, so Task 4 won't show on camera by itself. Tasks

1 to 3 will show easily, though: mushroom colors, the 4-hit kill, and sparks. For Task 4, you could record a separate short clip, or temporarily shorten the centipede in a local edit you don't commit. Say if you want that tweak.

Final checklist

- Commit order: baseline, Task 1, Task 2, Task 3, Task 4, then the after video, chat export and cleanup.
- Put everything in **Lab-4/** in your own repo, and don't raise a PR to SETAPESU26.
- Export the chat as PDF or doc, since the handout asks for that.

10/7/2026, 12:24:38 PM